Mergesort

Divide and conquer · Algorithms § 2.2

This video

- Why we need something faster than insertion sort
- **Merging**: combining two sorted halves into one (the part to really get)
- **The recursion**: how mergesort uses merge
- Why mergesort is n log n

Exactly how much slower are $O(n^2)$ algorithms?

growth rate	**n = 1,000**	**n = 1,000,000**
n log n	10,000	20,000,000
n^2	1,000,000	1,000,000,000,000

Selection sort and insertion sort are both **n^2**.

Mergesort is **n log n**.

On a million items: about **1 second** for an n log n sort, and about **14 hours** for an n^2 sort.

Mergesort top-level overview

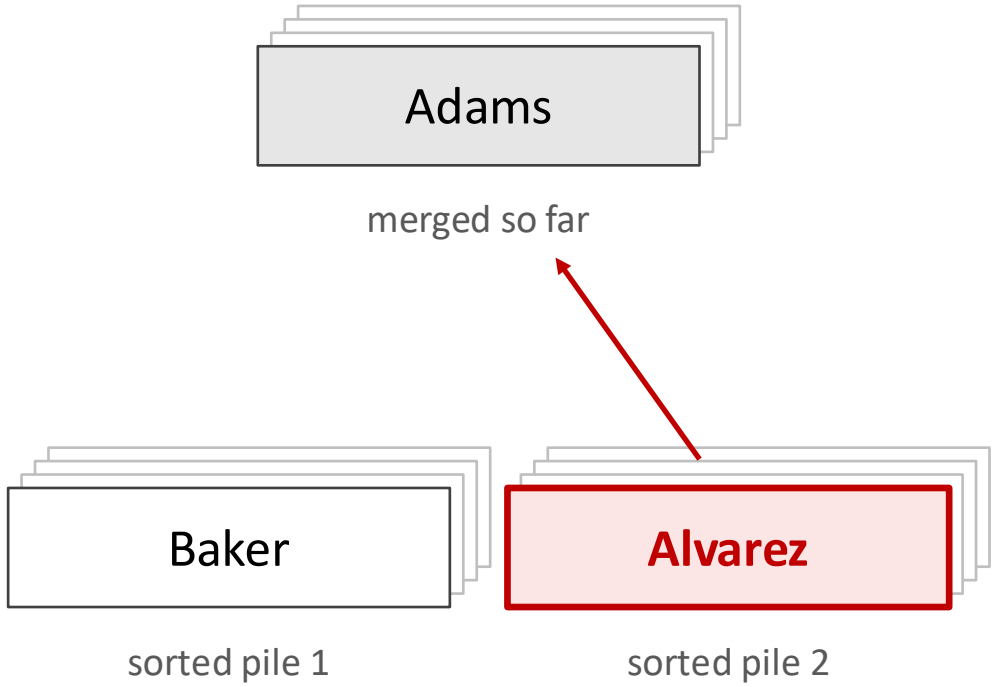
You have a stack of 200 exams to put in alphabetical order.

One at a time: take each exam and slide it into place among the ones you've already sorted. That's insertion sort, and with a big stack it takes forever.

Split it instead: split the stack in two, and sort each pile.

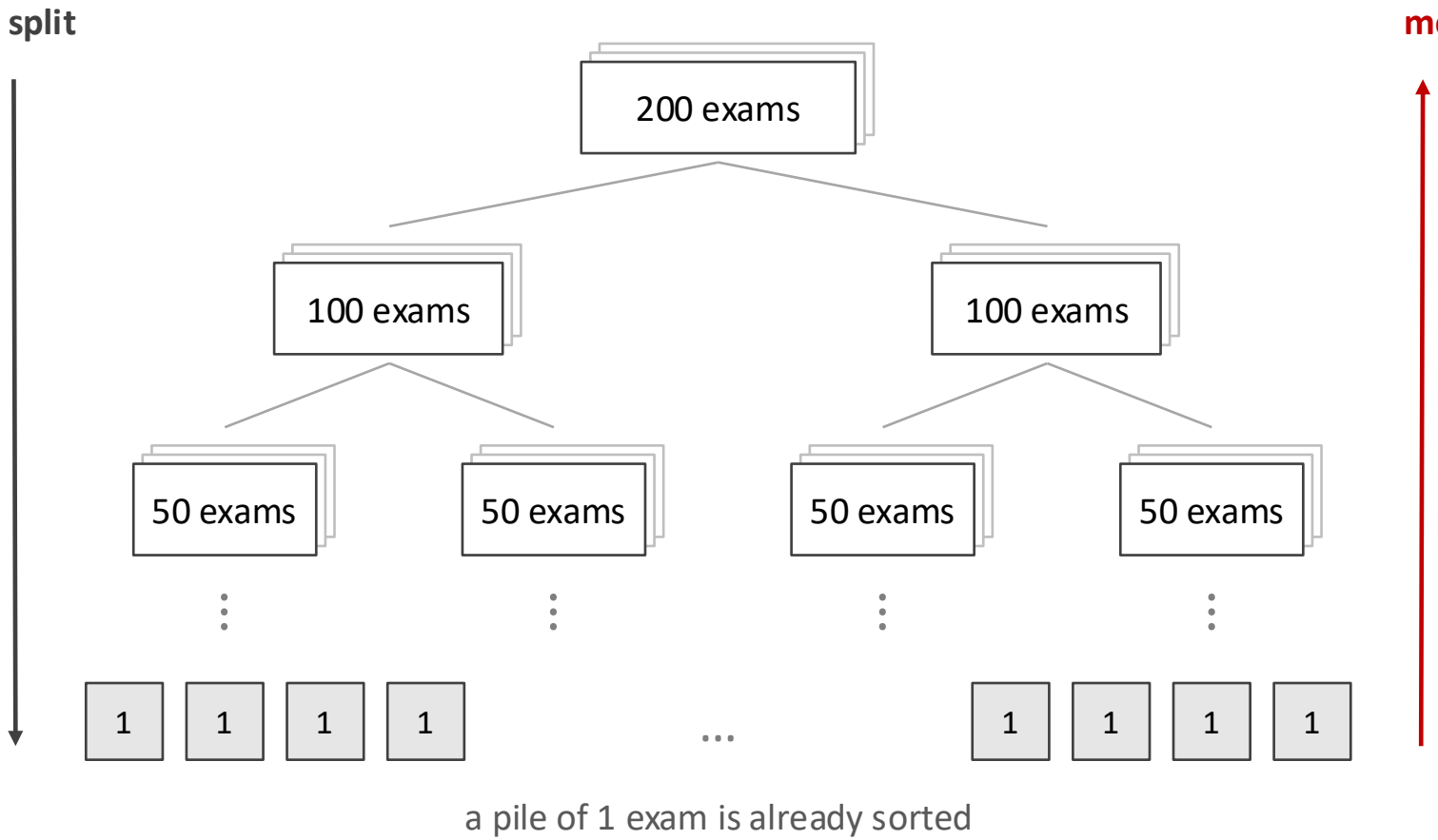
Then merge: look at the top exam on each pile, and move whichever name comes first onto a new pile. Repeat until both piles are empty.

THAT'S IT!: It doesn't look like this is a full solution to the problem, but thanks to recursion it *is*.



Merging is easy: you only ever compare the **two top** exams. Alvarez comes before Baker, so Alvarez goes next.

How do you sort the two piles? The same way.



a pile of 1 exam is already sorted

Base case: a pile of 1 exam is already sorted.

Recursive call: split the pile, sort each smaller pile **the same way**, then merge.

Base case: one exam

1 exam

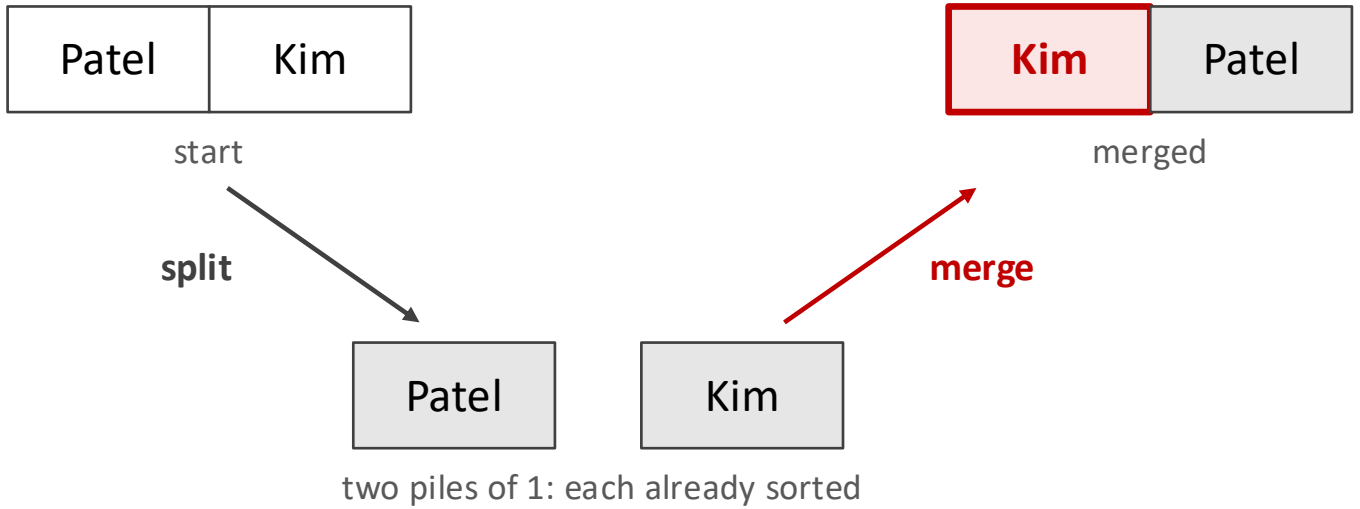


Already in order.

Nothing to do.

This is the **base case**.

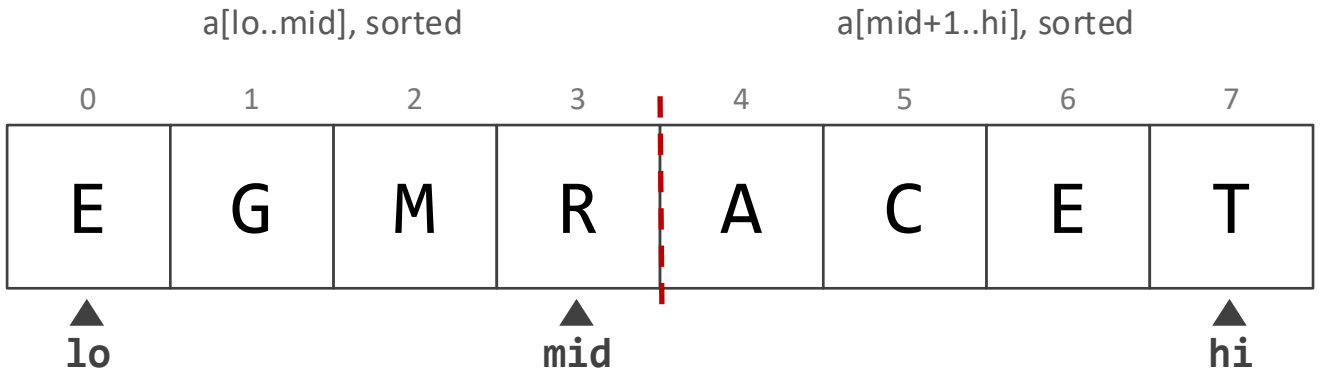
2 exams



Merge: is Kim before Patel? **Yes**, so Kim goes first. **One comparison.**

That's all of mergesort, at its smallest size. Every bigger stack does the same thing: **split down** to single exams, then **merge back up**.

What does it look like in an array?

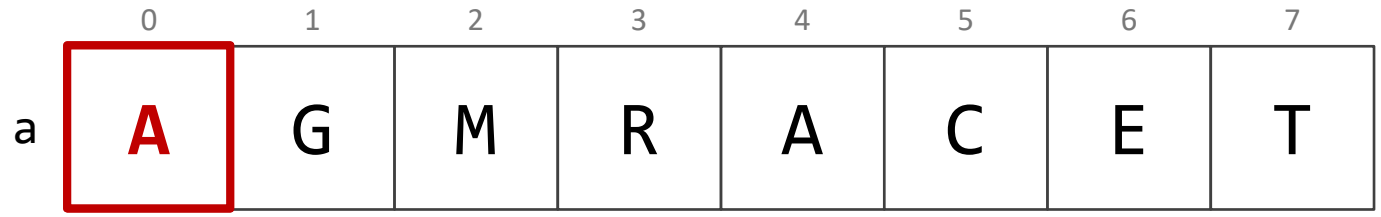


Given: two halves, `a[lo..mid]` and `a[mid+1..hi]`, each already sorted.

Goal: all of `a[lo..hi]` sorted.

The key fact: the smallest item overall is at the front of one of the two halves. So: compare the two front items, take the smaller, and repeat.

Why we need a second array, aux

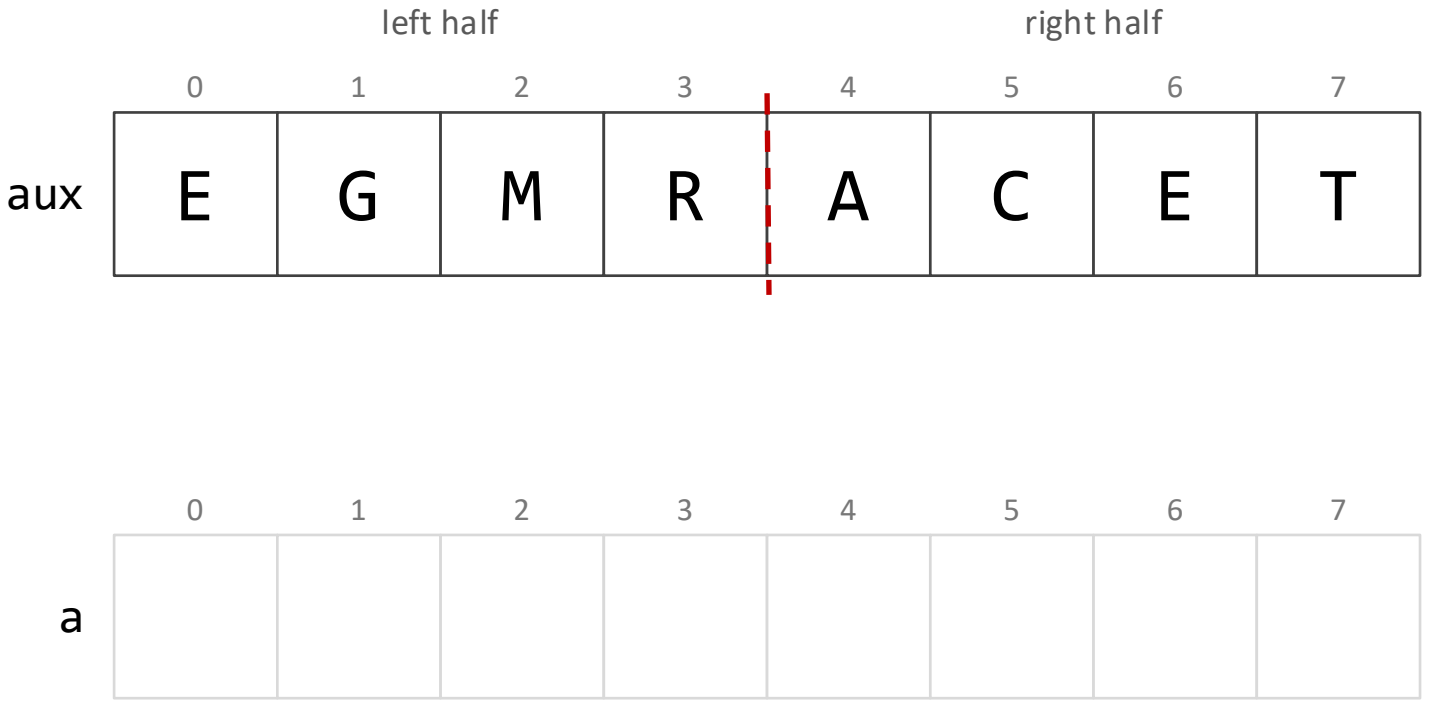


Put the smallest item, **A**, into `a[0]` and **E** is overwritten before we've placed it. It's gone.

The fix: first copy `a[lo..hi]` into a second array, `aux`. Then merge **from** `aux` **back into** `a`.

`aux` is created once, in the public `sort` method, and passed along. That part's already written for you.

Merge: trace



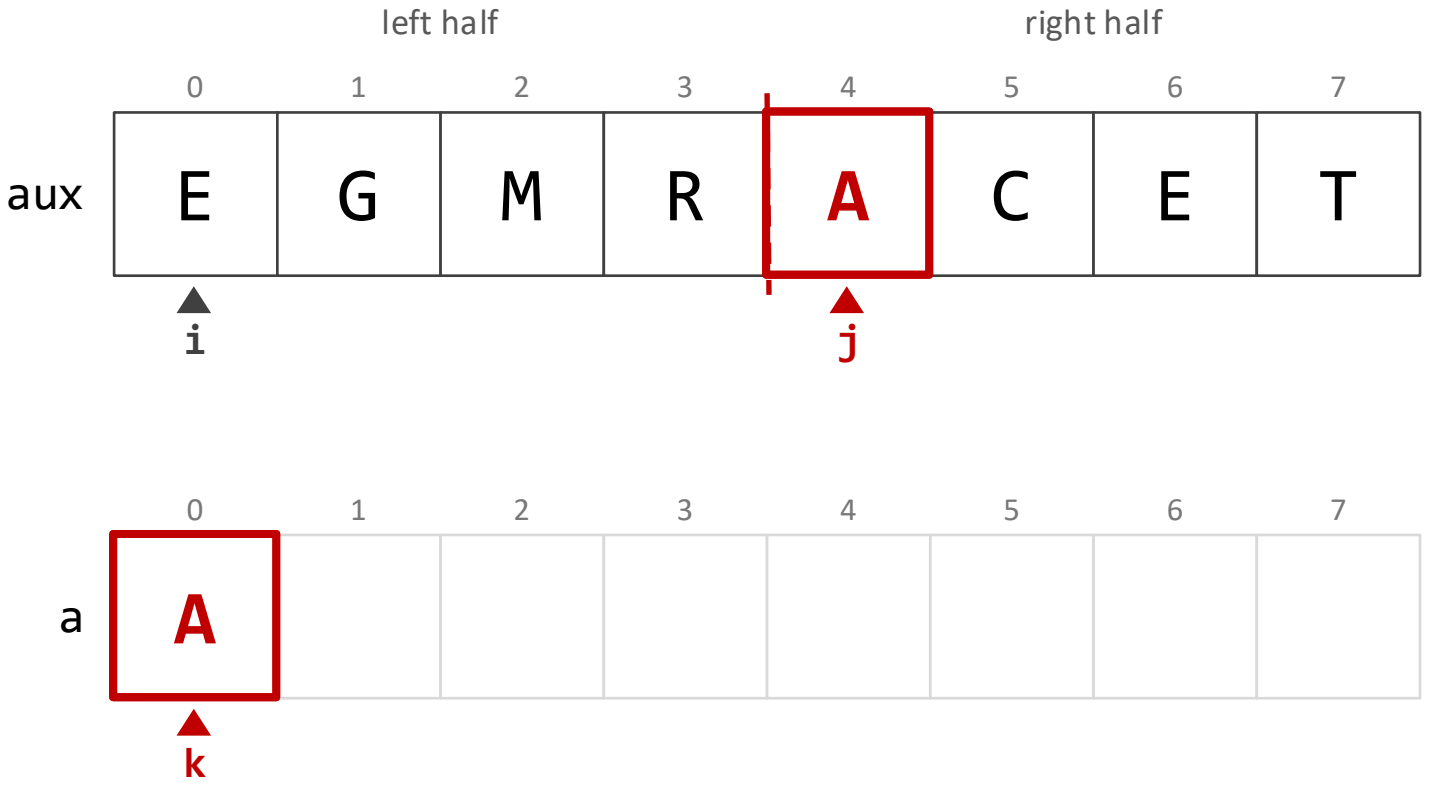
cost so far

compares	0
copies	8

8 copied into aux

Step 1: copy `a[lo..hi]` into `aux[lo..hi]`. Now we fill the original array (a) back in, left to right.

Merge: trace



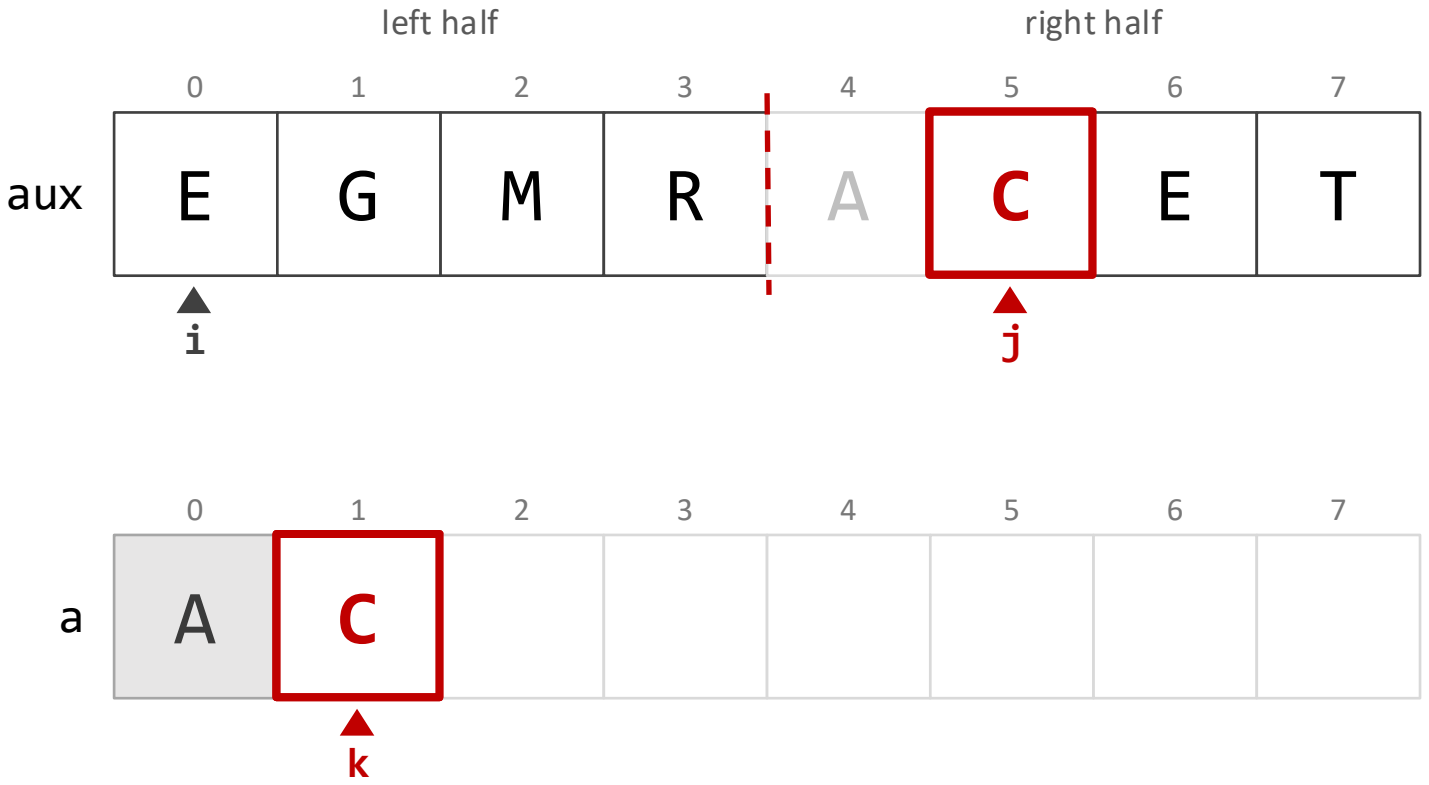
cost so far

compares	1
copies	9

8 into aux, 1 back into a

k = 0. Is A less than E? **Yes**: take **A** from the right.

Merge: trace



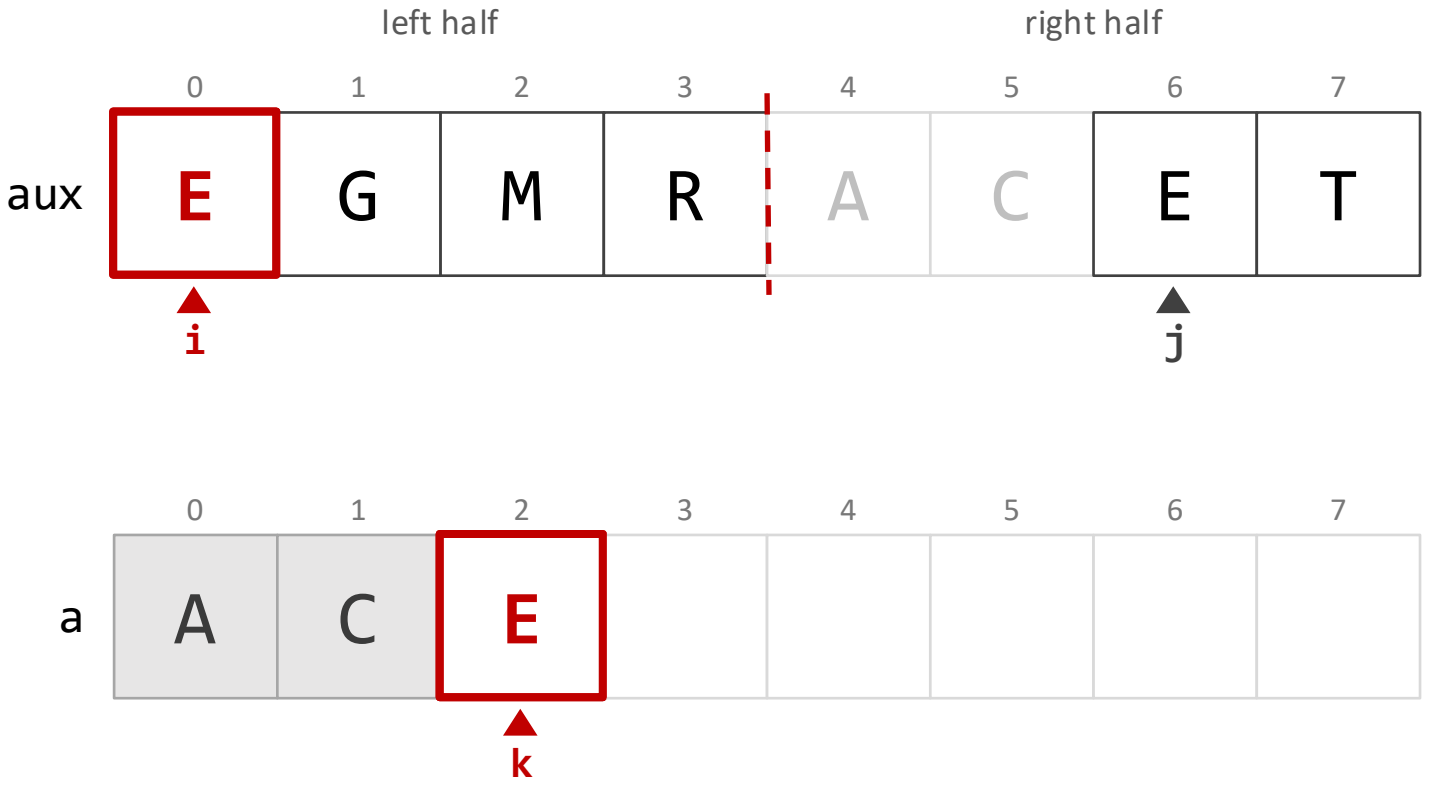
cost so far

compares	2
copies	10

8 into aux, 2 back into a

k = 1. Is C less than E? **Yes**: take **C** from the right.

Merge: trace



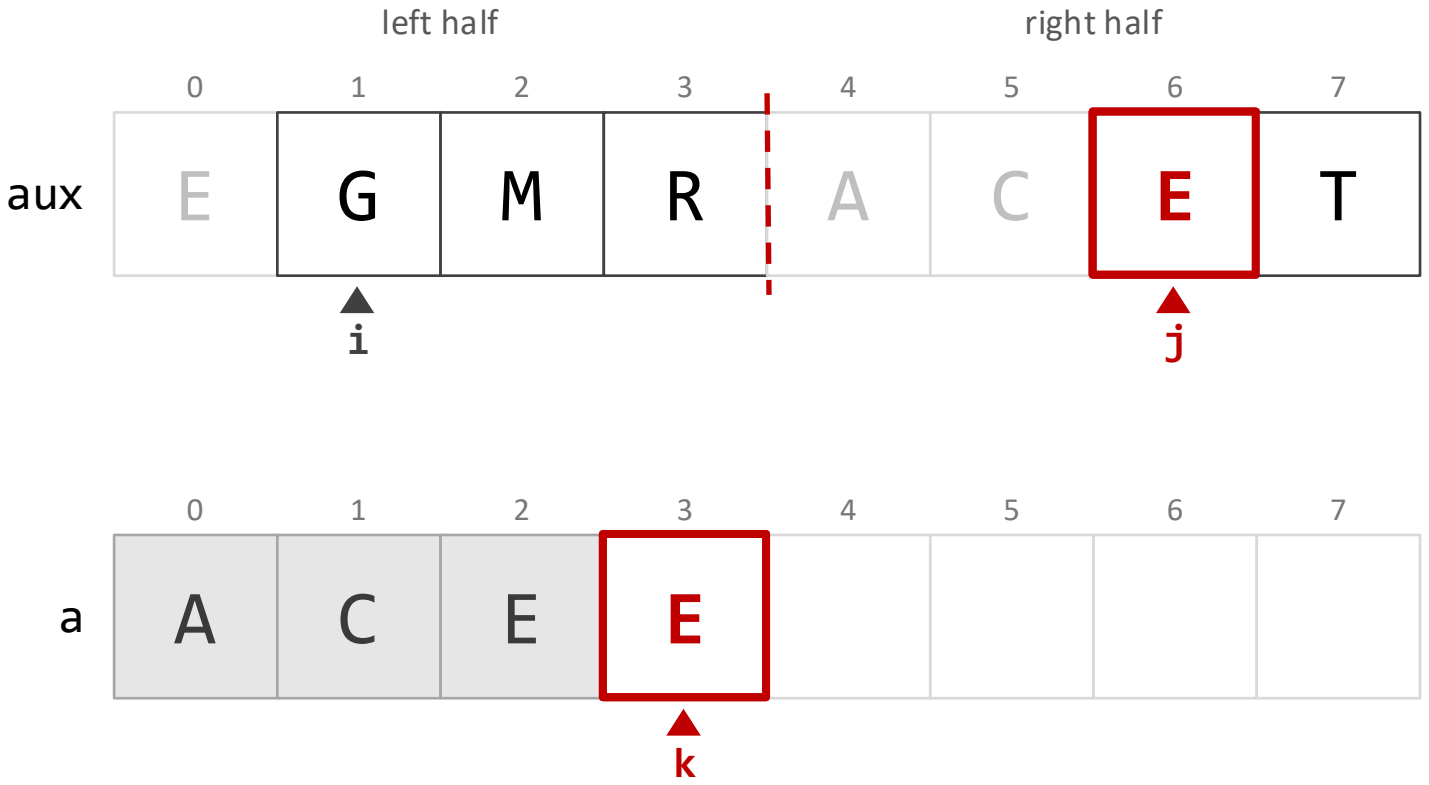
cost so far

compares	3
copies	11

8 into aux, 3 back into a

k = 2. Is E less than E? **No**, they're equal. On a tie, take **E** from the **left**.

Merge: trace



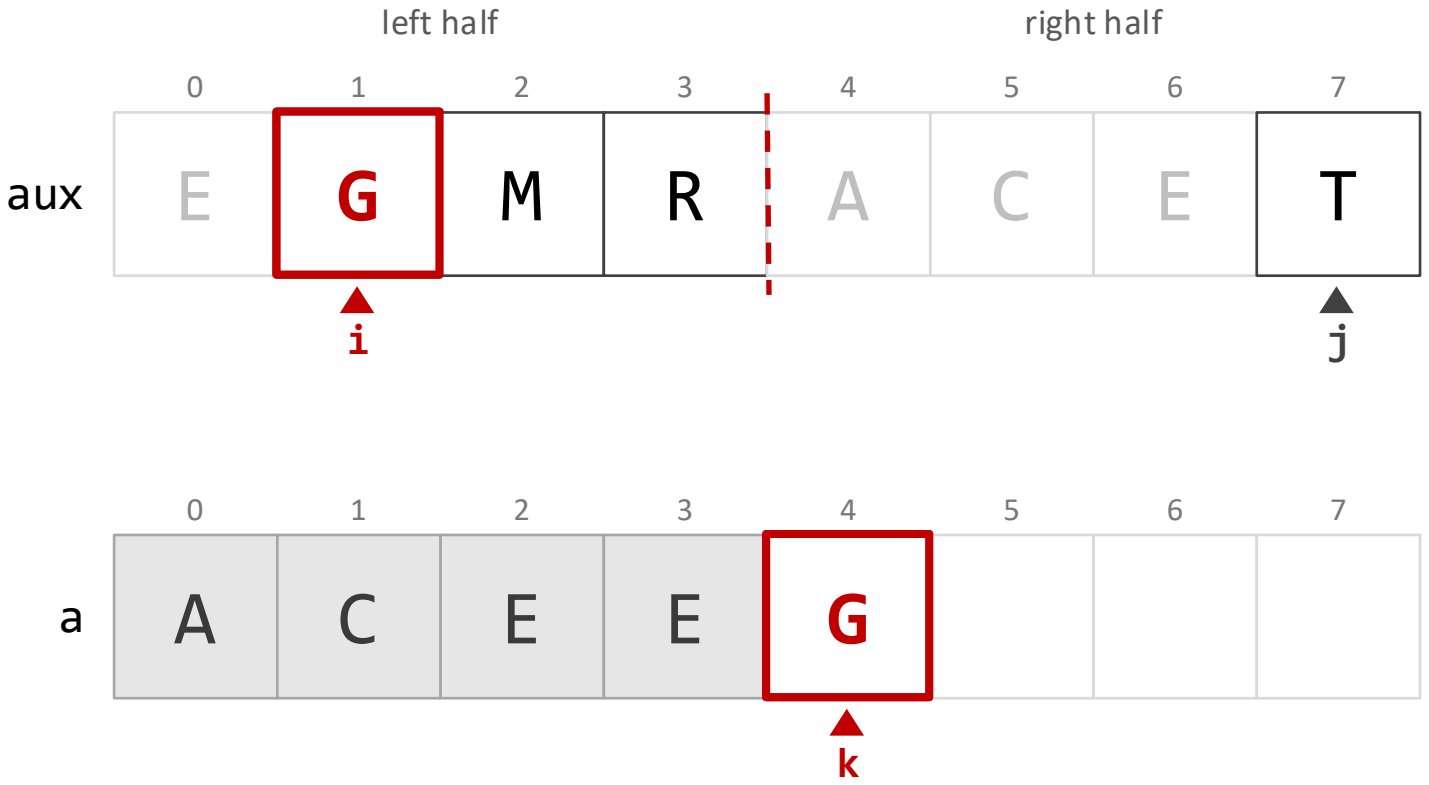
cost so far

compares	**4**
copies	**12**

8 into aux, 4 back into a

k = 3. Is E less than G? **Yes**: take **E** from the right.

Merge: trace



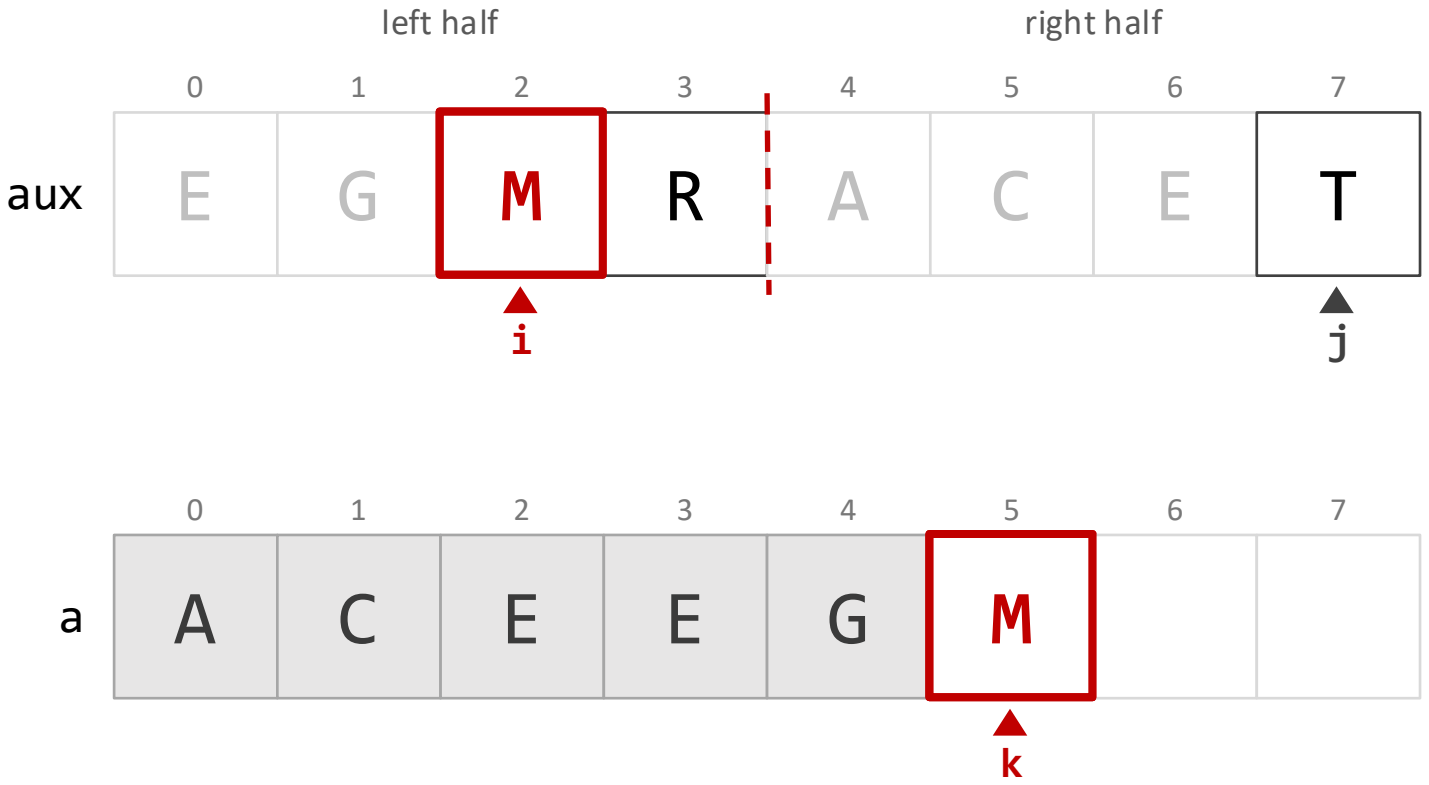
cost so far

compares	**5**
copies	**13**

8 into aux, 5 back into a

k = 4. Is T less than G? **No**: take **G** from the left.

Merge: trace



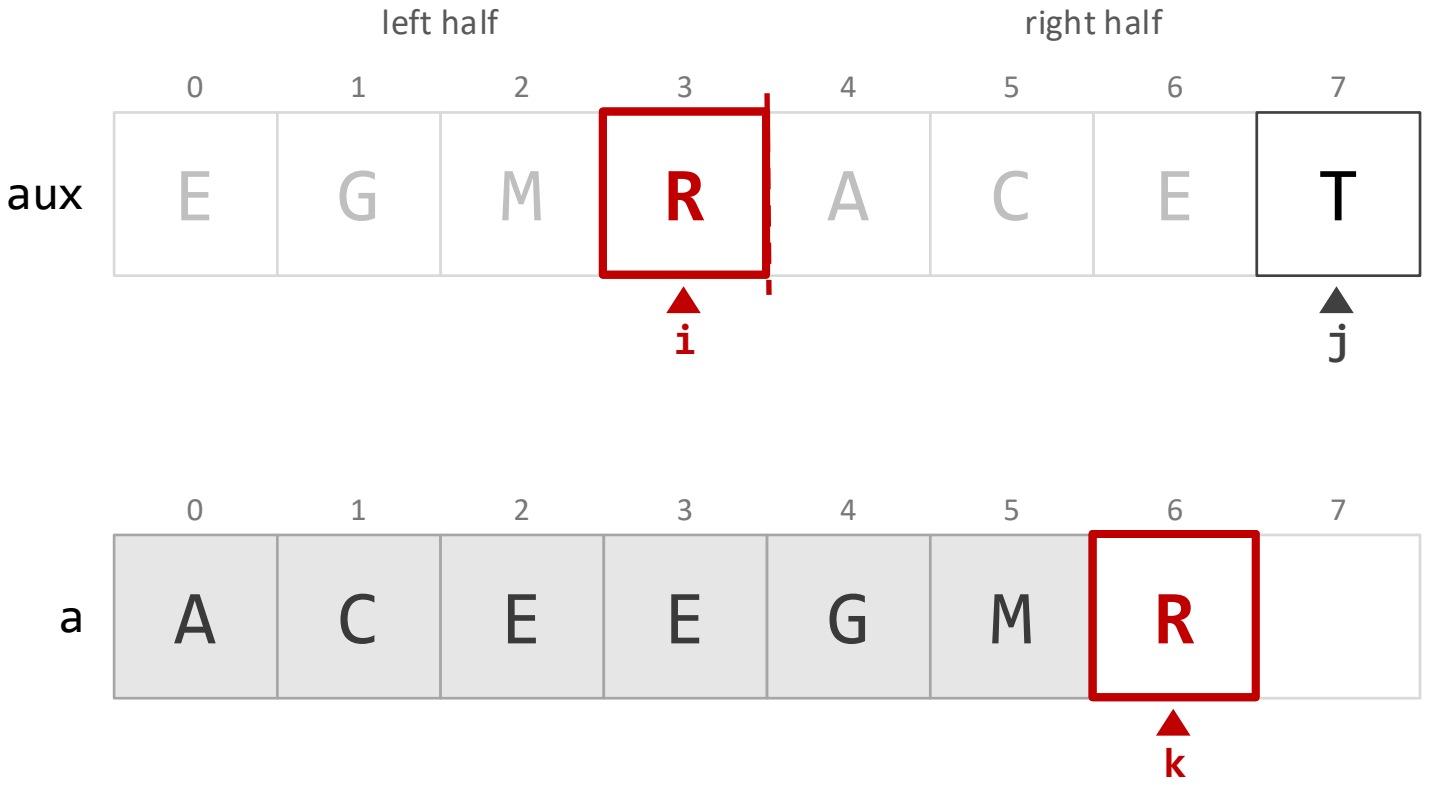
cost so far

compares	**6**
copies	**14**

8 into aux, 6 back into a

k = 5. Is T less than M? **No**: take **M** from the left.

Merge: trace



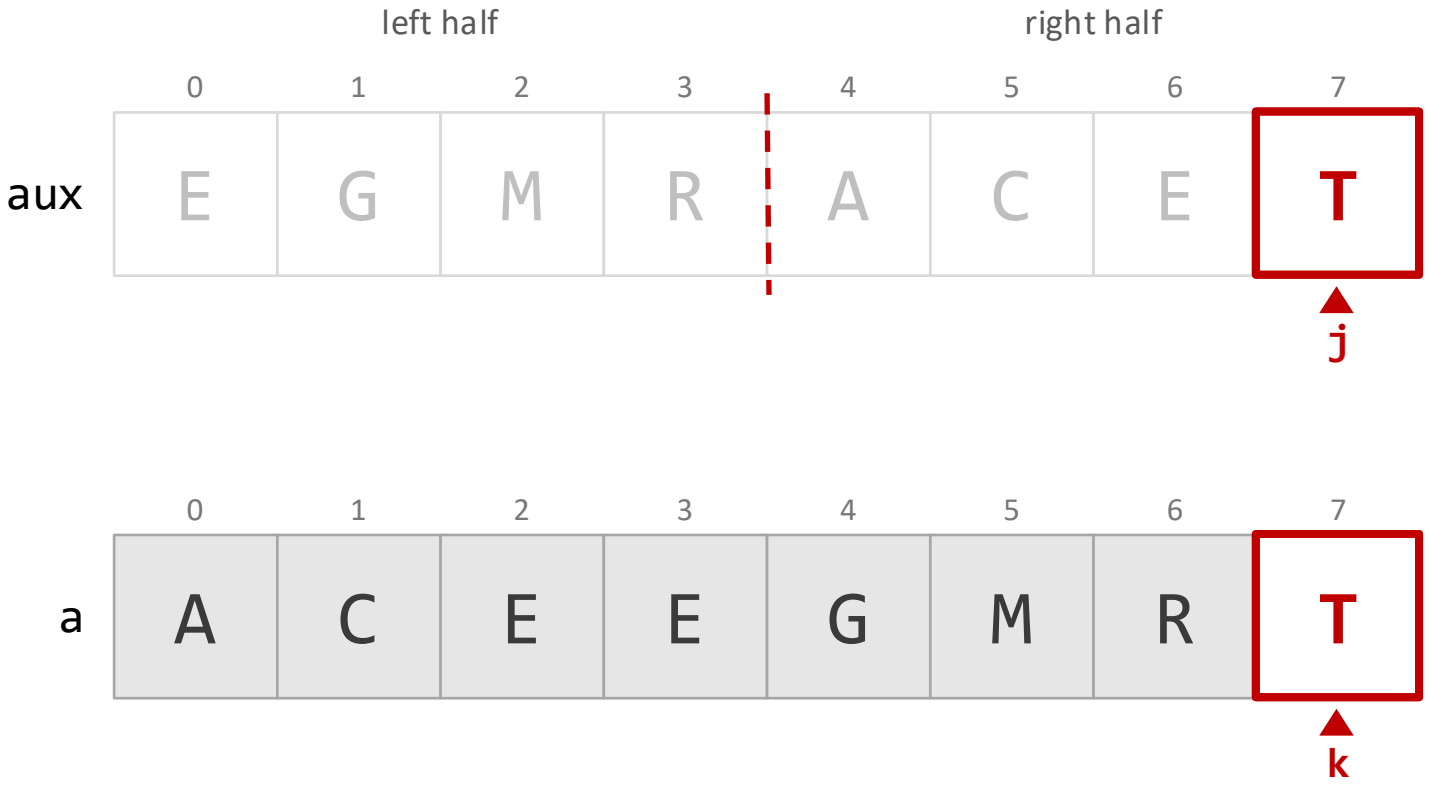
cost so far

compares	**7**
copies	**15**

8 into aux, 7 back into a

k = 6. Is T less than R? **No**: take **R** from the left.

Merge: trace



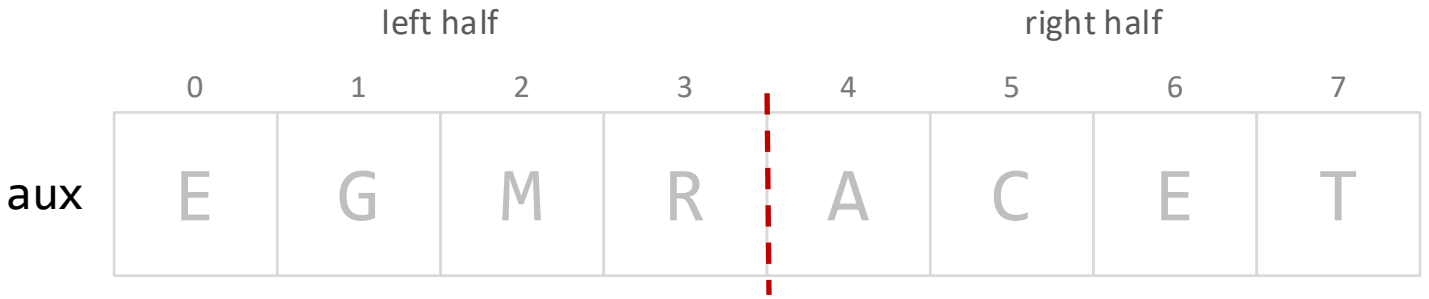
cost so far

compares	7
copies	**16**

8 into aux, 8 back into a

k = 7. The left half is **used up** (i > mid). Take **T** from the right.

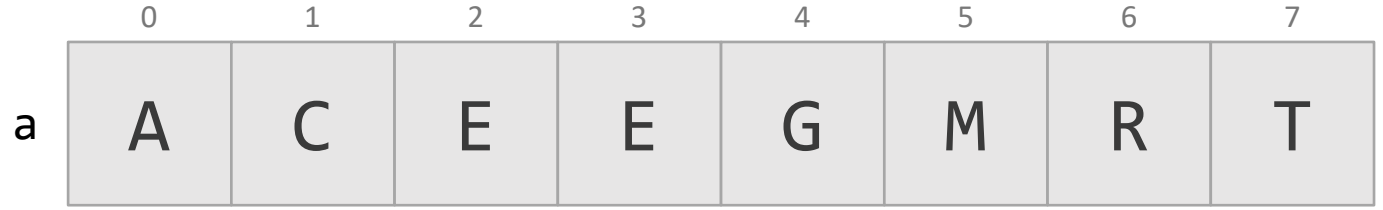
Merge: trace



cost so far

compares	7
copies	16

8 into aux, 8 back into a



Done. `a[lo..hi]` is sorted. 8 items placed, using **7 compares** and **16 copies**.

Merge: pseudocode

```
1  copy a[lo..hi] into aux[lo..hi]
2  i = lo
3  j = mid + 1
4  for k from lo to hi:
5      if the left half is used up (i > mid):       a[k] = aux[j], then j = j + 1
6      else if the right half is used up (j > hi): a[k] = aux[i], then i = i + 1
7      else if aux[j] is less than aux[i]:          a[k] = aux[j], then j = j + 1
8      else:                                        a[k] = aux[i], then i = i + 1
```

The order of the four cases matters.

- Check for a **used-up half first** (lines 5–6). Otherwise you'd compare against an item past the end of a half.
- The last case takes from the **left**, so on a tie the left item goes first. That keeps equal items in their original order, which makes mergesort **stable**.

How many compares does a merge take?

- Each compare places **exactly one** item into a.
- Once one half is used up, the rest of the other half is copied with **no** compares.
- The very last item placed never needs a compare.

So merging n items takes **at most n – 1 compares**. In the trace above, 8 items took 7.

The recursion: trivial base case + recursive call

1. Base case: one item (or none) is already sorted, so just return.

2. Recursive calls that get closer to the base case: `sort(a, aux, lo, mid)` and `sort(a, aux, mid + 1, hi)` each sort a range about **half the size**.

Trust them. `sort` promises that when it returns, `a[lo..hi]` is sorted. Assume both calls keep that promise. Then all that's left is to **merge** the two sorted halves.

Shaky on recursion?

Read the "survival recursion" slides. They build up to this exact shape on a much simpler problem.

Mergesort: pseudocode

```
1  sort(a, aux, lo, hi):
2      if hi <= lo: return              (one item or none)
3      mid = lo + (hi - lo) / 2
4      sort(a, aux, lo, mid)            (sort the left half)
5      sort(a, aux, mid + 1, hi)        (sort the right half)
6      merge(a, aux, lo, mid, hi)       (merge the two halves)
```

Each line becomes **one line of Java**.

`mid = lo + (hi - lo) / 2` is the middle of `lo..hi`, rounded down. (Java's integer division does the rounding.)

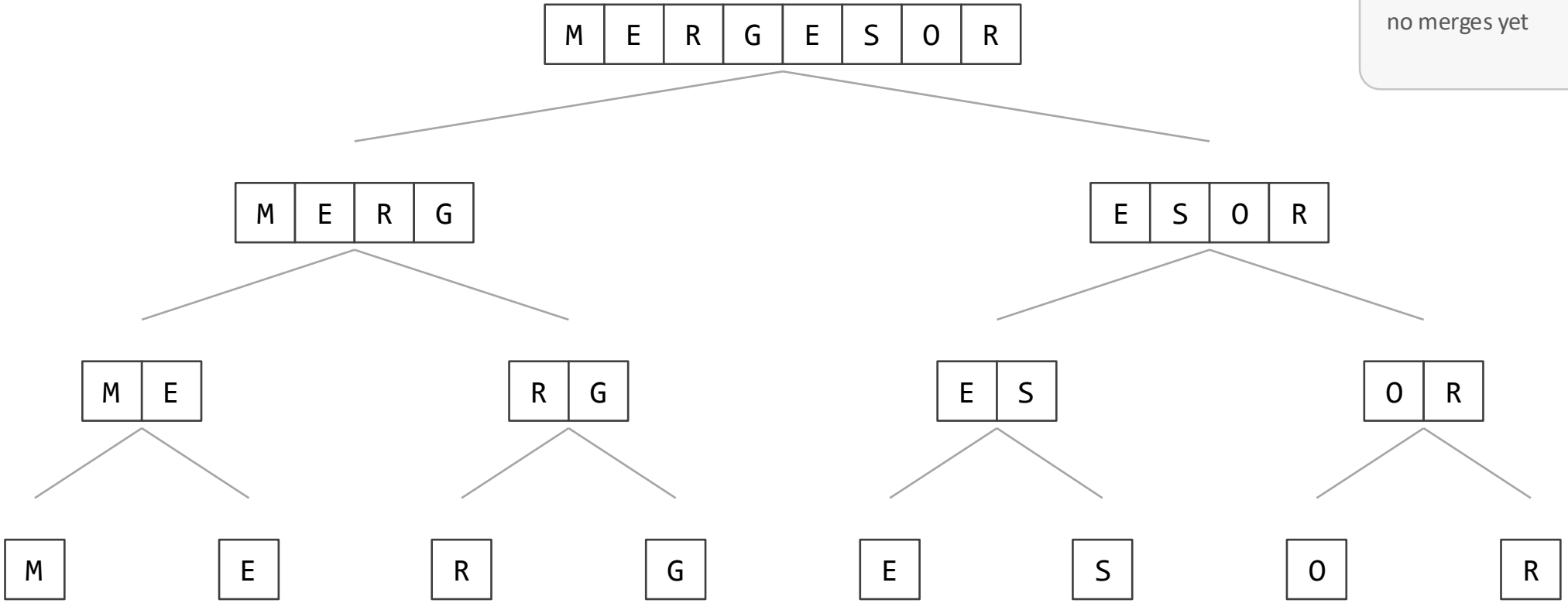
The right half starts at `mid + 1`, **not** `mid`. Starting at `mid` puts the middle item in both halves, and the recursion never ends.

Mergesort: the calls

cost of the whole sort so far

compares	0
copies	0

no merges yet



Each box is one call to `sort`. The calls split the array until every piece is a **single item**, which is already sorted.

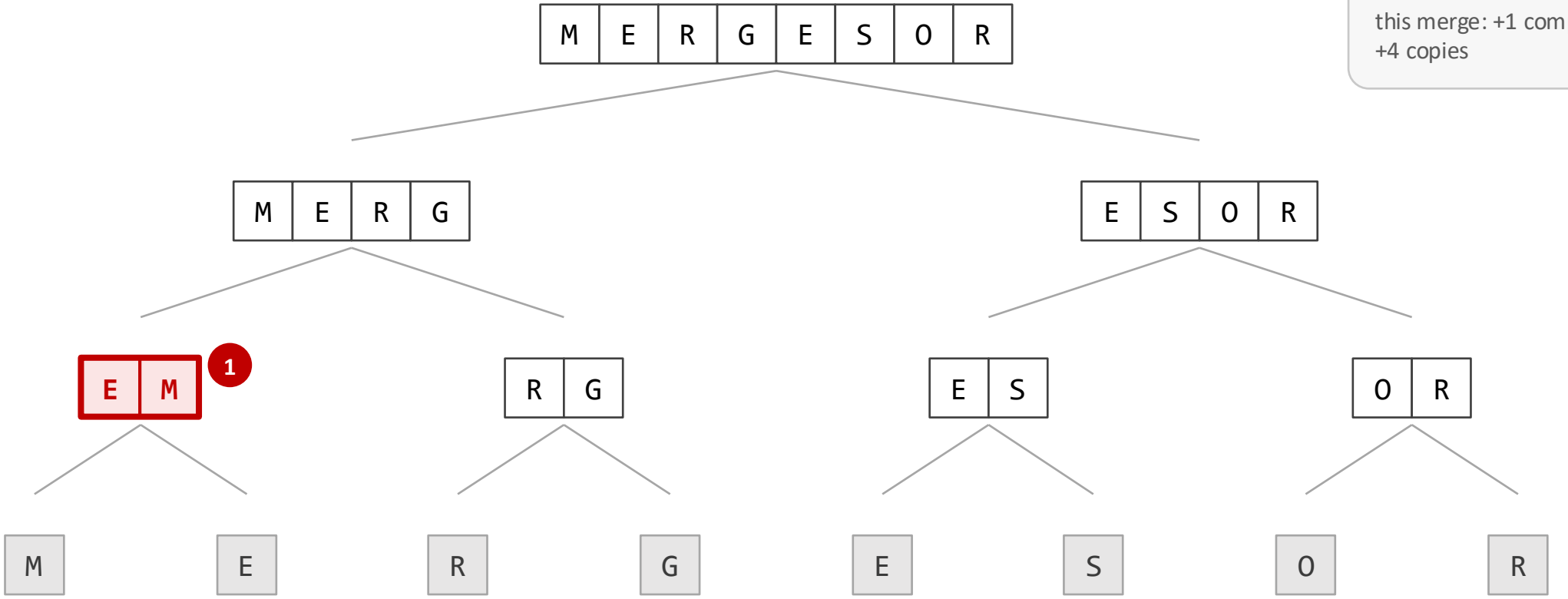
Mergesort: the calls

cost of the whole sort so far

compares **1**

copies **4**

this merge: +1 compares, +4 copies



Merge 1: `merge(a, aux, 0, 0, 1)` makes `a[0..1]` = E M

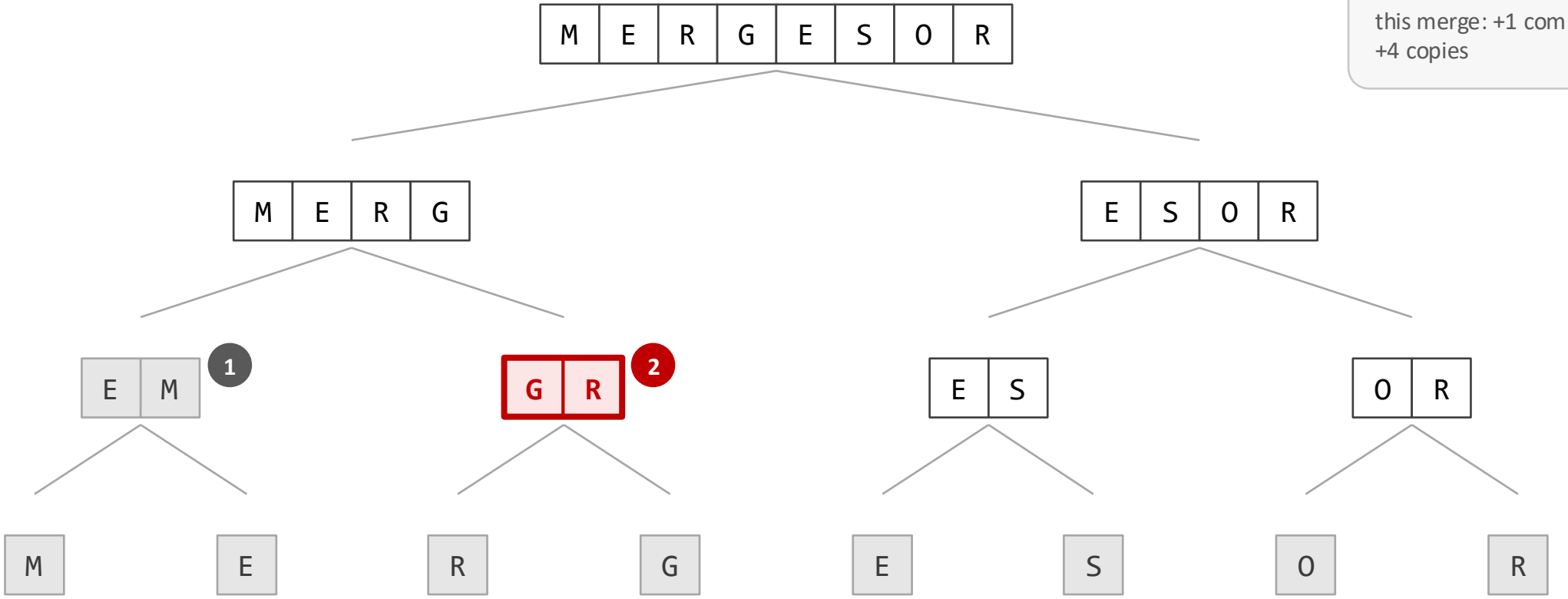
Mergesort: the calls

cost of the whole sort so far

compares **2**

copies **8**

this merge: +1 compares, +4 copies



Merge 2: `merge(a, aux, 2, 2, 3)` makes `a[2..3]` = G R

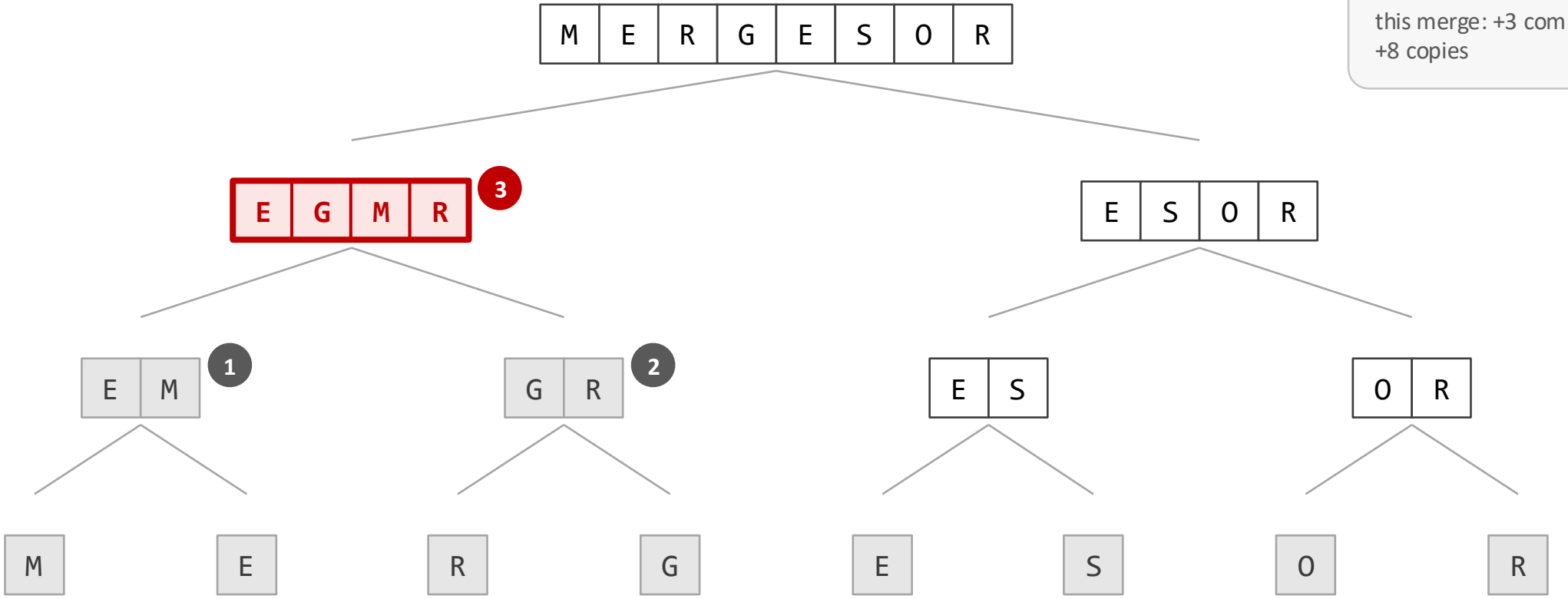
Mergesort: the calls

cost of the whole sort so far

compares **5**

copies **16**

this merge: +3 compares, +8 copies



Merge 3: `merge(a, aux, 0, 1, 3)` makes `a[0..3]` = E G M R

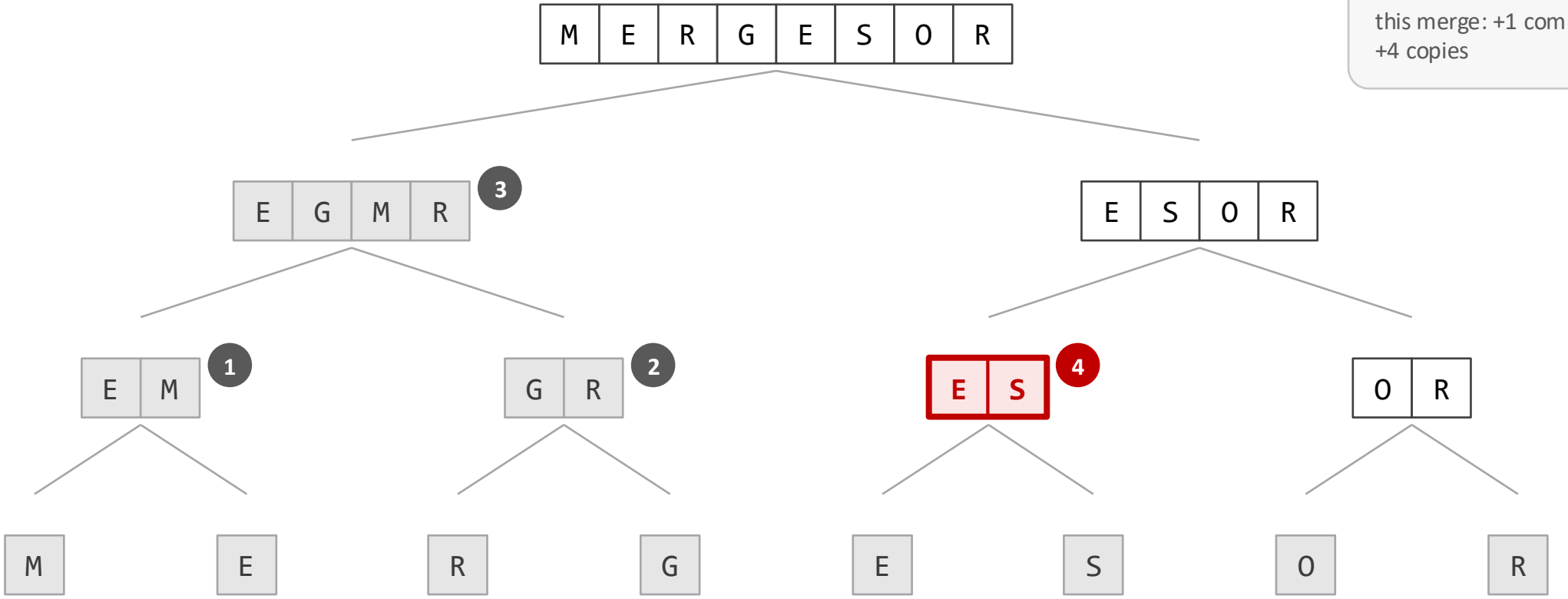
Mergesort: the calls

cost of the whole sort so far

compares **6**

copies **20**

this merge: +1 compares, +4 copies



Merge 4: `merge(a, aux, 4, 4, 5)` makes `a[4..5]` = E S

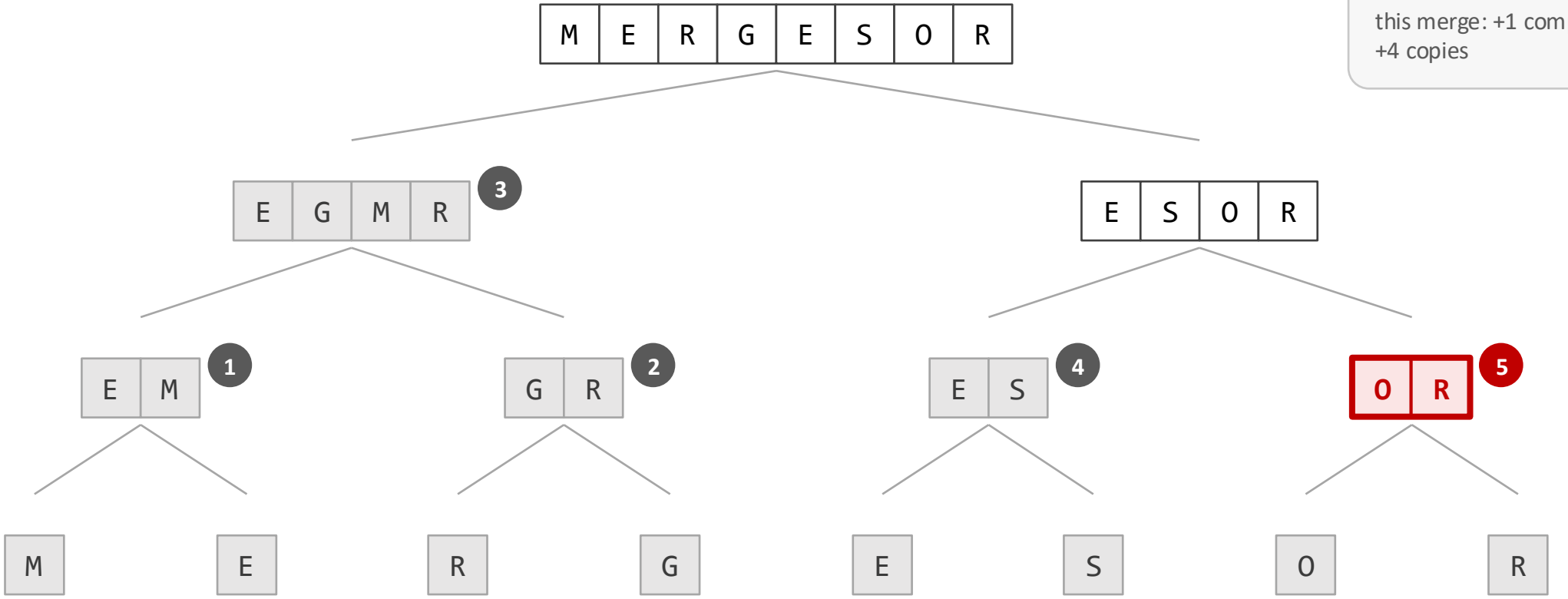
Mergesort: the calls

cost of the whole sort so far

compares **7**

copies **24**

this merge: +1 compares, +4 copies



Merge 5: `merge(a, aux, 6, 6, 7)` makes `a[6..7]` = O R

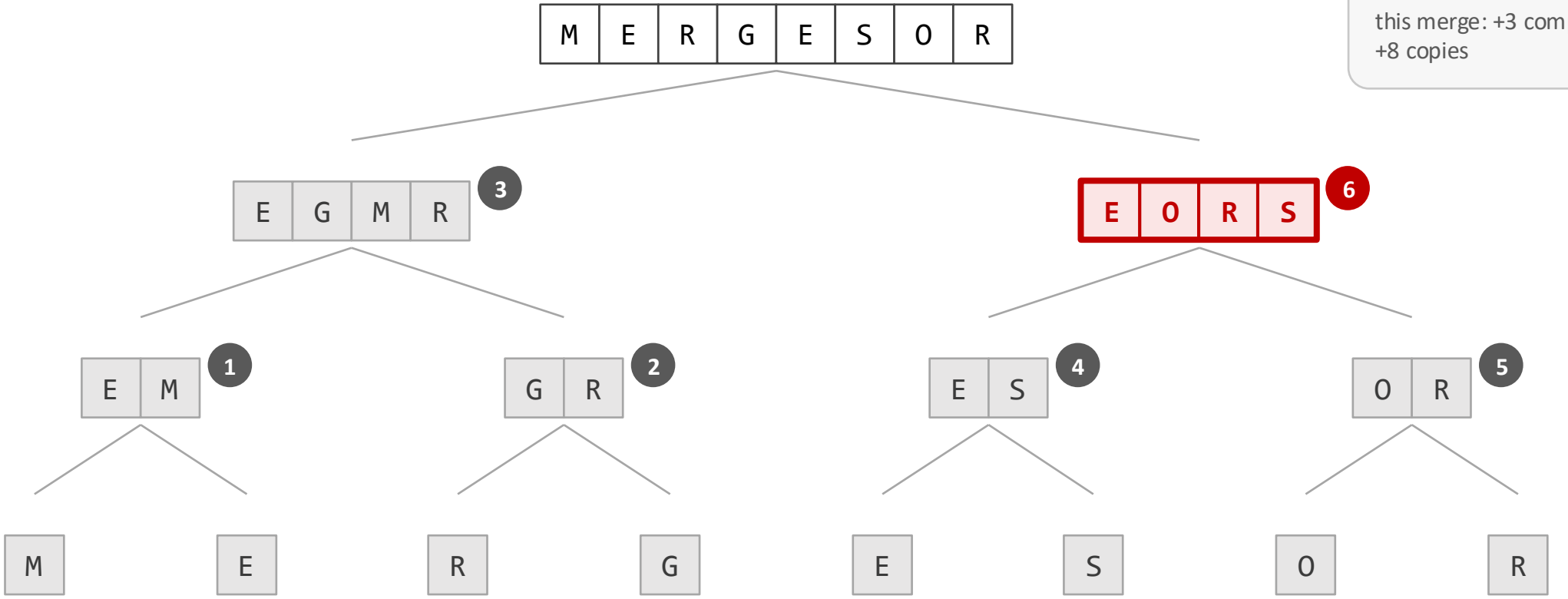
Mergesort: the calls

cost of the whole sort so far

compares **10**

copies **32**

this merge: +3 compares, +8 copies



Merge 6: `merge(a, aux, 4, 5, 7)` makes `a[4..7]` = E O R S

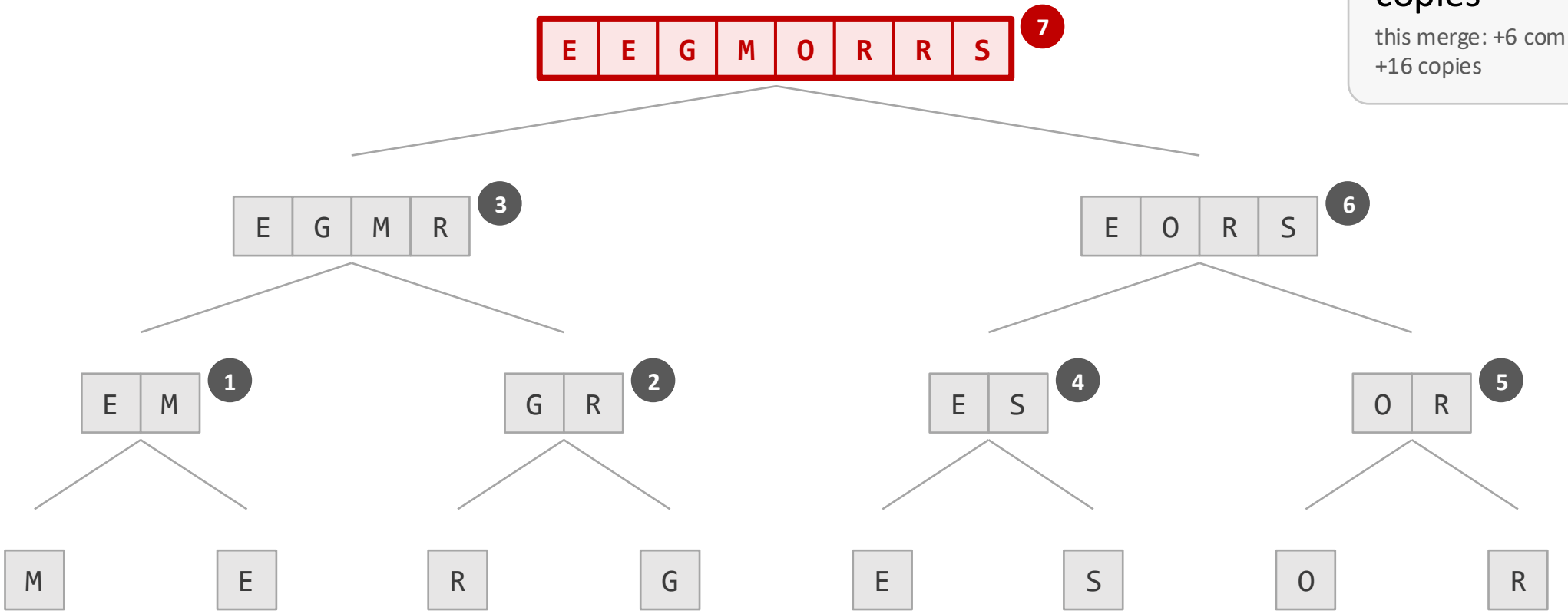
Mergesort: the calls

cost of the whole sort so far

compares **16**

copies **48**

this merge: +6 compares, +16 copies



Merge 7: `merge(a, aux, 0, 3, 7)` makes `a[0..7]` = E E G M O R R S

Why mergesort is n log n

level	**pieces**	**items merged at this level**
1	1 piece of n	n
2	2 pieces of n/2	n
3	4 pieces of n/4	n
…	…	…
$\log_2 n$	n/2 pieces of 2	n

Every level merges **n** items in total, so it makes at most **n** compares.

Halving each time means there are only $\mathbf{\log_2 n}$ levels. (From the big-O slides: about a million items is only 20 halvings.)

So: at most $\mathbf{n \log_2 n}$ compares.

The book's Proposition F: mergesort uses between ½ n lg n and n lg n compares.

Insertion sort vs. mergesort, as n grows

n	insertion sort	mergesort	insertion sort makes…
8 (M E R G E S O R)	15	16	**about the same**
800 (random)	160,817	6,722	**24× as many**
8,000,000 (random)	≈ 16,000,000,000,000 *	173,391,663	**≈ 92,000× as many**

compares, counted by running each sort

At a billion compares per second, sorting 8 million items takes insertion sort about **4½ hours**, and mergesort about **0.17 seconds**.

But look at the first row: on **tiny** inputs, insertion sort is just as good. That's why real-world sorts switch to insertion sort once the pieces get small.

* Estimated as $n^2/4$, too slow to actually run. At 800 items, the measured count (160,817) matched $n^2/4$ (160,000) to within 1%

The trade-off

	insertion sort	**mergesort**
compares, random input	~$n^2/4$	≤ n lg n
1,000,000 items	hours	about a second
extra memory	none	aux: n items
stable?	yes	yes (ties go left)

Mergesort's price is the extra array. **Next week:** quicksort, which sorts in place, with no aux.

Why it matters that mergesort is stable

A sort is **stable** if equal items stay in the **same order they started in**.

1. Sorted by name

name	section
Adams	2
Baker	1
Chen	2
Diaz	1
Evans	2
Fox	1

2. Then by section: stable sort

name	section
Baker	1
Diaz	1
Fox	1
Adams	2
Chen	2
Evans	2

Each section is still in **alphabetical order**.

3. Then by section: unstable sort

name	section
Diaz	1
Baker	1
Fox	1
Adams	2
Evans	2
Chen	2

Sections are right, but names in red are **out of alphabetical order** within their section.

Mergesort is stable (ties go left), and so is insertion sort. **Next week: quicksort** is usually faster, but it **isn't** stable. That's one reason Java's `Arrays.sort` uses a mergesort-based algorithm for objects.